C200 Programming Assignment №5

Professor M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

February 16, 2024

Introduction

Due Date: 11:00 PM, Friday, February 23 Please note that both Autograder submission `https://c200.luddy.indiana.edu/` and pushing to GitHub **must be done before the deadline**. As mentioned in the Inscribe post, we will be checking for cheating. If this results in a grade change, we are compelled by the University to send a formal notification to the Dean.

Instructions

1. Please check that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Kindly ensure that the **code does not have infinite loop (that never exits)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Computing Love

To match people, companies use trigonometry. Assume you have a list of people $[p_0, p_1, \ldots, p_m]$, we find the two different people who have the smallest angle. The way we calculate this is to assume each person is a list (mathematically vector) of 0s and 1s. We need three basic functions: inner product, magnitude, and $\cos^{-1}$. We assume two lists $x = [x_0, x_1, \ldots, x_n], y = [y_0, y_1, \ldots, y_n]$ of 0s and 1s of the same length:

$$inner_prod(x, y) = x_0y_0 + x_1y_1 + \cdots + x_ny_n \quad (1)$$

$$mag(x) = \sqrt{inner_prod(x, x)} \quad (2)$$

For the last function (where we calculate the angle), we know that:

$$\cos(\theta) = \frac{inner_prod(x, y)}{mag(x)mag(y)} \quad (3)$$

In class we learned that we can invert a function and the inverted function is denoted as f^{-1}. So we can:

$$\cos^{-1}(\cos(\theta)) = \cos^{-1}(\frac{inner_prod(x, y)}{mag(x)mag(y)}) \quad (4)$$

$$\theta = \cos^{-1}(\frac{inner_prod(x, y)}{mag(x)mag(y)}) \quad (5)$$

The math module has math.acos() for $\cos^{-1}()$. Further, Python returns θ in radians. We have:

$$\pi \text{ radians} = 180 \text{ degrees} \quad (6)$$

Thus, to convert from radians to degree, you **must** multiply your answer by $\frac{180}{\pi}$.

Your task is to first complete the functions “inner_prd“, “mag“ and “angle“ and then use them to write the “match“ and “best_match“ functions. The match function takes a list of people pi where each person is a list of 0s 1s, and returns all unique pairs with the angle in degrees. Note that “match“ returns the output in the format: [[person 1, person2, angle], [person2, person3, angle], [person1, person3, angle]], where each person i.e., person1, person2-is also a list for example, [1,1,1] or [0,1,0]. So it returns a list of lists. For “angle“ function-round your answer to 2 decimal digits.

```
people0 = [[0,1,1],[1,0,0],[1,1,1]]
print(match(people0))
print(best_match(match(people0)))
```

gives an output

```
[[[0, 1, 1], [1, 0, 0], 90.0],
 [[0, 1, 1], [1, 1, 1], 35.26],
 [[1, 0, 0], [1, 1, 1], 54.74]]
([0, 1, 1], [1, 1, 1], 35.26)
```

We're displaying the result of match so you can see the structure. Each unique pair as an angle. For example:

$$
\begin{aligned}
inner_prod([1,0,0],[1,1,1]) &= 1(1)+0(1)+0(1)=1 && (7)\\
mag([1,0,0]) &= \sqrt{inner_prod([1,0,0],[1,0,0])}=\sqrt{1}=1 && (8)\\
mag([1,1,1]) &= \sqrt{inner_prod([1,1,1],[1,1,1])}=\sqrt{3} && (9)\\
\theta &= (\frac{180}{\pi})\mathrm{math.acos}(\frac{1}{1\sqrt{3}})\approx 54.74 && (10)
\end{aligned}
$$

Deliverables for Problem 1

- Complete the functions.
- Note that function angle() utilizes equation 5.
- Round the output of angle to two decimal places.

Problem 2: Maximal Value

In this problem, you're given a list of numbers. The function msi takes a list of numbers and returns a list of three values (in this same order) namely, the interval start, interval stop and the value of the greatest sum. For example if the list is: x = [7, -9, 5, 10, -9, 6, 9, 3, 3, 9] then the greatest interval is from x[2:10] giving 36:

$$sum(x[2,10]) \quad = \quad 5+10-9+6+9+3+3+9=36 \tag{11}$$

The function should return a list of values i.e., [2, 10, 36]

```
x =  [7, -9, 5, 10, -9, 6, 9, 3, 3, 9]
print(msi(x))
```

produces

```
[2,10,36]
```

For this problem you are allowed to use sum(lst) which returns the sum of a non-empty list of numbers.

Programming Problem 2: Maximal Value

- Complete the function msi(x)
- You are allowed to use the sum function
- You might find the following from the lecture useful:

```
>>> x = [1,2,3,-2]
>>> sum(x[1:3])
5
```

Problem 3: Predator-Prey Model

The Lotka-Volterra predator-prey model chacterizes two populations of animals–in this example, we'll assume the predator is a fox and the prey is a rabbit. We examine the total populations of each as they interact. For this, I'll write $r[t], f[t]$ to mean the populations of rabbit and fox respectively at $t = 0, 1, 2, 3, \ldots, k$. We're interested in the following, given $r[t], f[t]$ how do we compute $r[t+1], f[t+1]$? The models states that we must include four rates:

- The growth rate of the rabbit population. We'll use b_r for birth of rabbits.

- The death rate of rabbits due to fox predation. We'll use d_r to indicate this value.
- The death rate of foxes, *e.g.*, bad hunting skills, disease, scarce food. We'll use d_f for this value.
- Then an odd rate: When foxes catch and consume rabbits, they have litters (kits). So there's a rate of turning rabbits into foxes. We'll call this b_f.

The model is:

$$r[t+1] = \lceil r[t] + r[t]b_r - r[t]f[t]d_r \rceil \tag{12}$$

$$f[t+1] = \lceil f[t] + b_f d_r r[t]f[t] - f[t]d_f \rceil \tag{13}$$

We're using subscripting for the lists. This means given values at locations r[t] and f[t] for rabbits and foxes, we can add new members at locations f[t+1], r[t+1]. So the equations simply make a list of values–the location of the variables in the list is the time.

```
def rabbit_fox(br,dr,df,bf,rabbit,fox,time_limit):
    i = 0
    history_rabbit = []
    history_fox = []
    while i < time_limit:
        history_rabbit.append(rabbit)
        history_fox.append(fox)
        #marvelous code
    return history_rabbit,history_fox

br = 0.03
dr = 0.0004
df = 0.25
bf = 0.11
rabbit = 3000 #initial population size
fox = 200 #initial population size
time_limit = 2000
history_rabbit, history_fox = rabbit_fox(br,dr,df,bf,rabbit,fox, ↩
    time_limit)
```

produces

```
0 3000 200
200 5118 18
400 4496 150
600 6713 42
800 4274 46
1000 6204 140
1200 5902 33
1400 4094 77
```

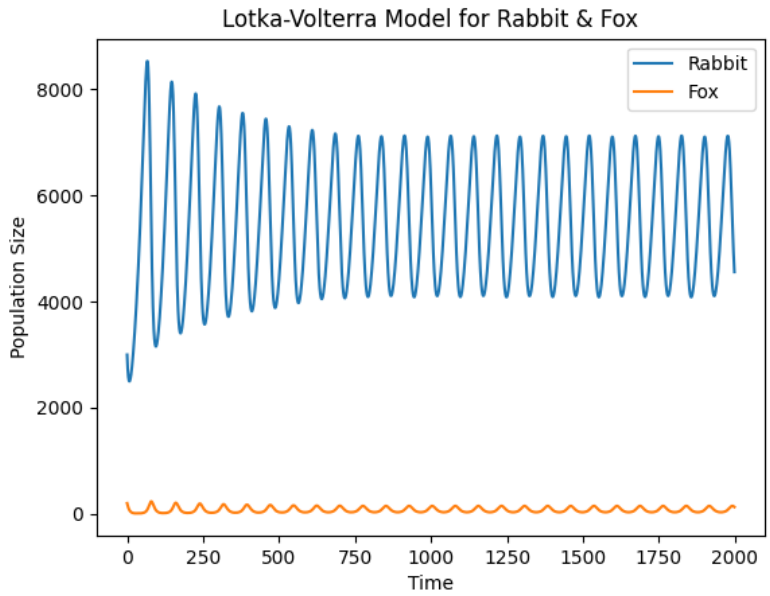

Figure 1: Predator-Prey Lotka-Volterra model for 2000 time points.

```
1600 7050 94
1800 5132 31
```

which describes the time point, rabbit population, fox population at some points and the graphic in Fig. 1. Your task is to complete the #marvelous code using equations 12, 13 and from inspection of the code. You can see the populations reach an equilibrium.

Deliverables for Problem 3

- Observe the ceiling function.
- You should experiement and see what are the populations constraints–does this *always* become stable?
- After testing the code but before submitting to the Autograder - make sure to comment lines 262-268 (the code used for creating the plot) because Autograder can not interpret visual output i.e. plots and will likely fail due to the code trying to display plots in the web browser. You are wlecome to use the plotting code while testing on your own computer.

Problem 4: Entropy

In this problem, we will be calculating what's called *Entropy*. Entropy is used in ML, AI, vision, finance, *etc.* Entropy is a single number that describes how random the data is. If it's 0, then the data isn't random. If there are n objects and each one is equally probable, then the entropy is maximal: $\log(n)$. How might this be useful? Decisions are "better" the less entropy there is.

For this problem, we are only working with finite probabilities. We can think of probabilities as a list of numbers $p_0, p_1, \ldots, p_n$ such that

$$\begin{aligned} p_i &\geq 0, \quad i = 0, 1, \ldots, n && (14) \\ 1 &= p_0 + p_1 + p_2 + \cdots + p_n && (15) \\ &= \sum_{i=0}^{n} p_i && (16) \end{aligned}$$

We can make a list (we'll assume the items are of the same type and immutable) into a probability. For example, consider a list `x = ["a","b","a","c","c","a"]`.

1. gather the items uniquely (*hint: dictionary*)
2. count each time the item occurs
3. create a new list of the `count/len(dictionary)`

The list of probabilities would be y = [3/6,1/6,2/6] (a's probability is 3/6, b's probability is 1/6 and c's probability is 2/6). Observe that if you add up these numbers, they will sum to one.

We still need to show you *how* to calculate entropy:

$$\begin{aligned} entropy &= -(p_0 \log_2(p_0) + p_1 \log_2(p_1) + \cdots + p_n \log_2(p_n)) && (17) \\ entropy(y) &= -(\frac{3}{6}\log_2(3/6) + \frac{1}{6}\log_2(1/6) + \frac{2}{6}\log_2(2/6)) && (18) \\ &= -(.5(-1.0) + 0.17(-2.58) + .33(-1.58)) && (19) \\ &= 1.46 && (20) \end{aligned}$$

Because of continuity arguments we treat $\log_2(0) = 0$. Python's math module will correctly state this math error, so you'll **have to simply add 0 if the probability is 0**.

Deliverables Problem 4

- Take a non-empty list of objects and calculate the entropy.
- You **cannot** use the `count` function.
- Complete the functions for this problem.
- Round the output of entropy to two decimal places.

Problem 5: Run of 1s

For a financial instrument like stock, we want to buy and sell at the lowest and highest values, respectively. By observing trends, better decisions about buying and selling can be made. A "trend" is when when a sequence of values is **monotonic**. A sequence of values $v_0, v_1, v_2, \ldots, v_n$ is increasingly monotonic if

$$v_0 \leq v_1 \leq \cdots \leq v_n$$

The increasing monotonicity allows for selling at a higher price. Conversely, there is decreasing monotonicity. In this problem we'll be writing code to look for increasing monotonicity, but simplifying the problem by using only zero or one. In this problem, you'll take a list of 0s and 1s as an input. Your program will output the *longest* sequential run of 1s (no 0s encountered).

Output function.py

```
L([1, 0, 1, 0, 1, 0])
1

L([0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0])
2

L([1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1])
4
```

Deliverables Problem 5

- Complete the function.

Problem 6: Nine of Cattails

It's known that if you add the digits of any number, and that it eventually equals nine, that the original number is divisible by nine. Here is an example:

$$39789 \quad \rightarrow \quad 3+9+7+8+9=36 \rightarrow 3+6=9$$

The order does not matter, *viz.,*

$$3+((9+7)+8)+9$$
$$3+((16)+8)+9$$
$$3+(7+8)+9$$
$$3+15+9$$
$$18+9$$
$$27 \rightarrow 9$$

We are left with only one digit which is 9 and hence the number 39789 is divisible by 9 (Infact, 9 * 4421 = 39789).

So we can see that the number 39789 is indeed divisible by 9 by following our method. Write a program that determines if a number is divisible by nine by using this method. You *cannot* simply divide by 9 and return True! You must continually add the numbers until a single digit is left; that digit will either be 9 or something else.

Caution: Remember that 'sum' is actually a built-in function in Python so we suggest that you store the results of addition in a variable named other than 'sum' to prevent errors or failures during testing.

sum = a + b + c (this is not a good practise because 'sum' is a reserved keyword in Python), rather my_sum = a + b + c (this is safe to do)

Deliverables Problem 6

- Complete the function **following** the algorithm described above.
- You can neither use the % sign in the program nor divide directly by 9.

Problem 7: Optimization of Cost

You're contracted to work with a trucking freight company We Like to Move it, Move it!. The hourly rates for drivers are $\$14\ per\ hour$ and $\$15\ per\ hour$. The operating cost of the truck is, in cents per mile,

$$truck(d, s) \quad = \quad (\frac{60+s}{2})d \tag{21}$$

where d, s are distance and speed, respectively. The total cost is then:

$$c(s, d, h) \quad = \quad driver(h, t) + truck(d, s) \tag{22}$$

where h is the hourly rate and t is the time for which the driver drove the truck. You inform the company that this is an optimization problem, given a fixed salary and distance. In particular, you say that since the speed must be from 40 mph to 60 mph, there is an optimal speed that reduces cost. Implement the functions that allow you to determine for a fixed distance and salary (we'll look at both), the speed that reduces cost. The following code:

```
s_a,s_b = 40,60
hr_rate = 14
distance = 100
print(min_cost(distance,hr_rate,(s_a,s_b)))
hr_rate = 15
print(min_cost(distance,hr_rate,(s_a,s_b)))
```

produces:

```
(82.92 53)
(84.77, 55)
```

One aspect of the problem you must pay attention to is the different units: dollars and cents. These cannot be added together, since they're different units, *e.g.*, 1 dollar plus 1 cent $\neq$ 2. Either you have 100 + 1 or 1 + .01.

Deliverables Problem 7

- Complete the functions **following** the algorithm described above.
- The return type of min_cost is a tuple (x, y) where x is the cost and y the speed. Round cost to 2 decimal places.
- Hint: Since, distance and speed are given so we can find the time (t) using those.

8: Logical Entailment

A simple logic function is a mapping $f : \{0,1\}^n \rightarrow \{0,1\}$. You know several, for example, AND is a logic function. One of the most important functions is *material implication*:

A	B	A $\rightarrow$ B
0	0	1
0	1	1
1	0	0
1	1	1

Table 1: Material Implication.

This reads, “If A, then B." The logic is sometimes hard to understand, so I’m giving you a simple characterization. Assume A means you’ve been poisoned (think of the *Princess Bride*). The poison is fatal unless you take an antidote B. The antidote is harmless if you’re not poisoned. The value of $A \rightarrow B$ is whether you’ll survive. Table 2 helps you understand this connective.

A	B	A $\rightarrow$ B	Survival
0	0	1	If I’m not poisoned and I don’t take the antidote, then I live!
0	1	1	If I’m not poisoned, but I take the antidote, then I live!
1	0	0	If I’m poisoned and I don’t take the antidote, then I die.
1	1	1	If I’m poinsoned, but I take the antidote, then I live!

Table 2: An interpretation of material implication.

Python does not have material implication (implication) as a native operator, so we have to implement the function. One approach is to observe there is only one pair of values that makes implication false, A=1, B=0. This is a conjunction and can be written as (A $\wedge\neg$ B). Since it makes implication false, we negate it to capture when implication is true:

$$\neg(\mathrm{A} \wedge \neg\mathrm{B}) \quad \leftrightarrow \quad \neg\mathrm{A} \vee \neg\neg\mathrm{B} \tag{23}$$

$$\leftrightarrow \quad \neg\mathrm{A} \vee \mathrm{B} \quad \text{De Morgan Rule} \tag{24}$$

So, we can implement material implication as a disjunction in Eq. 24. Observe that when $A = 0$, then the disjunction is true. When $A = 1$, the only time it’s false is when $B = 0$. So this matches the table.

A truth function f is a

- **tautology** if it’s true for all inputs
- **contradiction** if it’s false for all inputs
- **contingency** if it’s neither a tautology nor contradiction

You will implement a function kind(F) that takes a logic function of three inputs and determine whether it is a tautology, contradiction, or contingency. Look at the functions below:

$$S(P,Q,R) = (P \wedge (\neg Q \vee \neg R)) \rightarrow (P \rightarrow \neg Q) \tag{25}$$

$$T(A,B,C) = ((A \rightarrow B) \wedge (B \rightarrow C)) \rightarrow (A \rightarrow C) \tag{26}$$

$$U(E,F,G) = \neg T(E,F,G) \tag{27}$$

$$V(X,Y,Z) = U(X,Y,Z) \rightarrow S(X,Y,Z) \tag{28}$$

$$M(g,gg,ggg) = True \tag{29}$$

The following code:

```
print(kind(S))
print(kind(T))
print(kind(U))
print(kind(V))
print(kind(M))
```

produces:

```
contingency
tautology
contradiction
tautology
tautology
```

Deliverables Problem 8

- Complete the functions S, T, U,V and M as shown in equations 25, 26, 27, 28 and 29.
- Use the imp function to implement the material implication in the formulae.
- The return type of kind is a string.

9: While Loops and Roots

Remember that for a real-valued function $f(x)$, the root $x*$ is a value such that $f(x*) = 0$. We saw, for example, that $ax^2 + bx + c = 0$ has roots that can be analytically determined. Here is a very simple technique to discover roots for $f(x)$ if is a continuous function.

```
#initial estimate
x = alpha
#epsilon is how good we want the estimate
while abs(x - f(x)) > epsilon:
   x = f(x)
```

While it doesn't work all the time, for some problems, it does astonishingly well for such simple code. The main task is to find the reciprocal of the highest power. For example, assume $f(x) = x^2 - 4x - 7$. The highest power is x^2 so we isolate that:

$$0 = x^2 - 4x - 7 \tag{30}$$

$$x^2 = 4x + 7 \tag{31}$$

$$x = \sqrt{4x + 7} \tag{32}$$

We can quickly see that $f(4) < 0$ and $f(7) > 0$ so a root must lie between 4 and 7. So we start with 4 as our initial guess. The following code:

```
def f1(x):
    return math.sqrt(4*x + 7)

def approx_root(f,initial_guess):
    ...
x_star = approx_root(f1,4)
print(x_star, x_star**2 - 4*x_star - 7)
```

produces

```
5.3166247040232015 -5.726630121216658e-07
```

We know one of the roots is $\frac{4+\sqrt{16+28}}{2} \approx 5.3166247903554$ which is very close. We do one more example. Let $f(x) = x - e^{-x}$. Then

$$x = e^{-x} \tag{33}$$

Let's start with an initial guess as x = .5. Then

```
def f0(x):
    return math.exp(-x)

```

```python
def approx_root(f,initial_guess):
    ...
x_star = approx_root(f0,.5)
print(x_star, x_star - math.exp(-x_star))
```

produces

```
0.5671433381054858 7.474599861279074e-08
```

The other two functions' roots are:

$$f_2(x) = x^4 - 4x - 7 \tag{34}$$

$$f_3(x) = x^5 - 3x^2 - 2 \tag{35}$$

Deliverables for Problem 9

- Complete the functions shown in equations 34 and 35 and implement the approx_root function.
- You can verify your outputs if they're very, very small numbers.
- For checking if the output is small, we can check if it's less than $\epsilon(\epsilon = 1e^{-7})$.

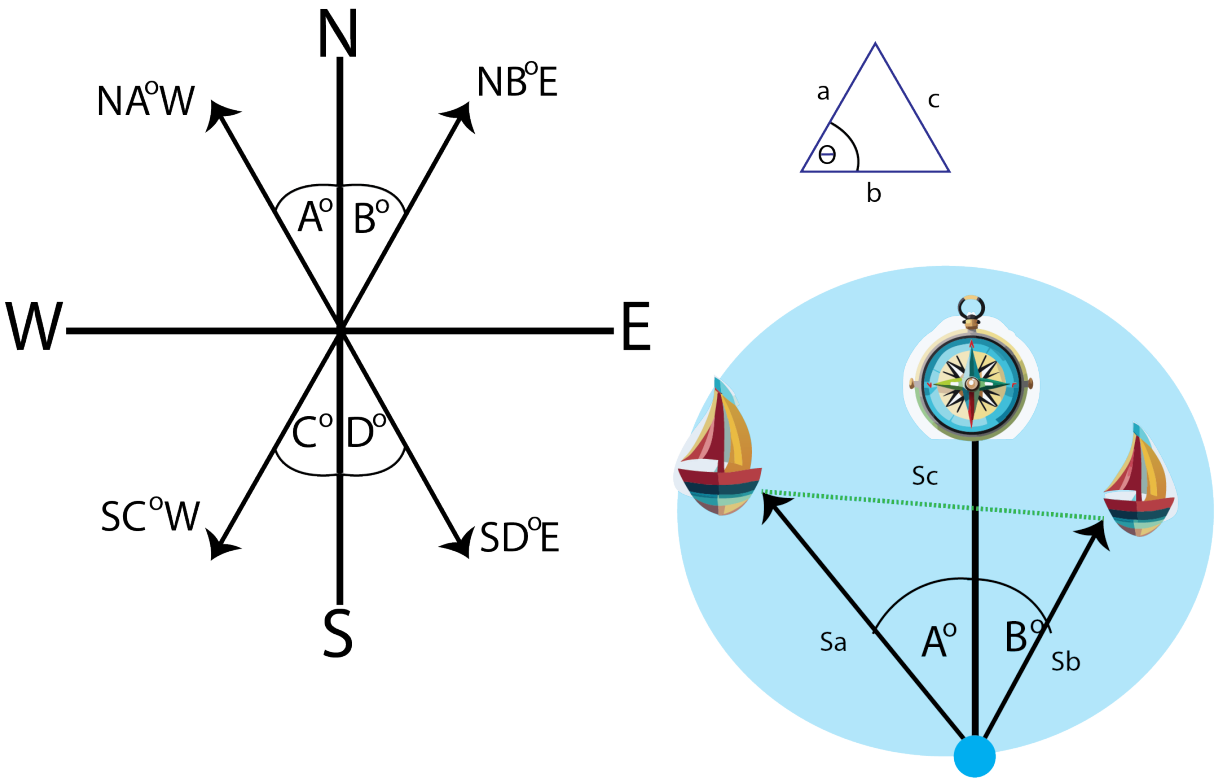

Figure 2: Upper left are the four bearings. Upper right is law of cosines triangle. Lower right are the two ships in our problem. If ship A has traveled Sa miles and B Sb miles, and the angle between is found from their respective bearings, then we can use the law of cosines to find Sc.

10: Sailing Away

In Fig. 2. the use of *bearing* is shown. In shipping, there are four directions used, called bearings, the trajectory of a vessle. Here is the BNF:

$$\langle Bearing \rangle \quad ::= \quad (\text{N}|\text{S})\langle degree \rangle(\text{E}|\text{W}) \tag{36}$$

$$\langle degree \rangle \quad ::= \quad 0.00|\ldots|360.00 \tag{37}$$

When moving north east, an acute angle is used from north moving clockwise. When moving north west, an acute angle is used from north moving counterclockwise. For south, this is repeated. For this problem, we'll write a general problem for two ships (Fig. 2. lower right). Assume one is headed bearing NX°E at some speed and a second is headed bearing NY°W at some speed. After some time t has elapsed, how far apart are the two ships? We can use the law of cosines, where a, b are the distances they've travelled individually and θ is the angle between, since we can simply sum the two angles from the bearing: For a triangle with the length of two sides a, b known and the angle θ they subtend, we can find the remaining length c as:

$$c^2 \quad = \quad a^2 + b^2 - 2ab \cdot cos(\theta) \tag{38}$$

The Fig 2. at the upper right has a triangle labeled.

$$\langle Bearing \rangle \quad ::= \quad (\mathsf{N}|\mathsf{S})\langle degree \rangle(\mathsf{E}|\mathsf{W}) \tag{39}$$

$$\langle degree \rangle \quad ::= \quad 0.00|\ldots|360.00 \tag{40}$$

You'll be implementing two functions, one for distance and one for the law of cosines. To test this problem:

- At 1:00P, ship A leaves the harbor bearning N34°W at 24 mph.
- At 1:30P, ship B leaves the harbor bearning N56°E at 18 mph.

Find the distance between the two ships, A and B, at 3:00P.

The following code:

```
Sa = 24
Sb = 18
start_a = 1
start_b = 1.5
stop = 3
a,b = 34,56

print(cosine_law(*(distance(start_a,stop,Sa),distance(start_b,stop,Sb)↩
    ),a+b))
```

produces

```
55.07
```

The *(distance(start_a,stop,Sa),distance(start_b,stop,Sb)) is sending a tuple of two distance values to the cosine_law function.

Deliverables Problem 10

- Complete the functions.

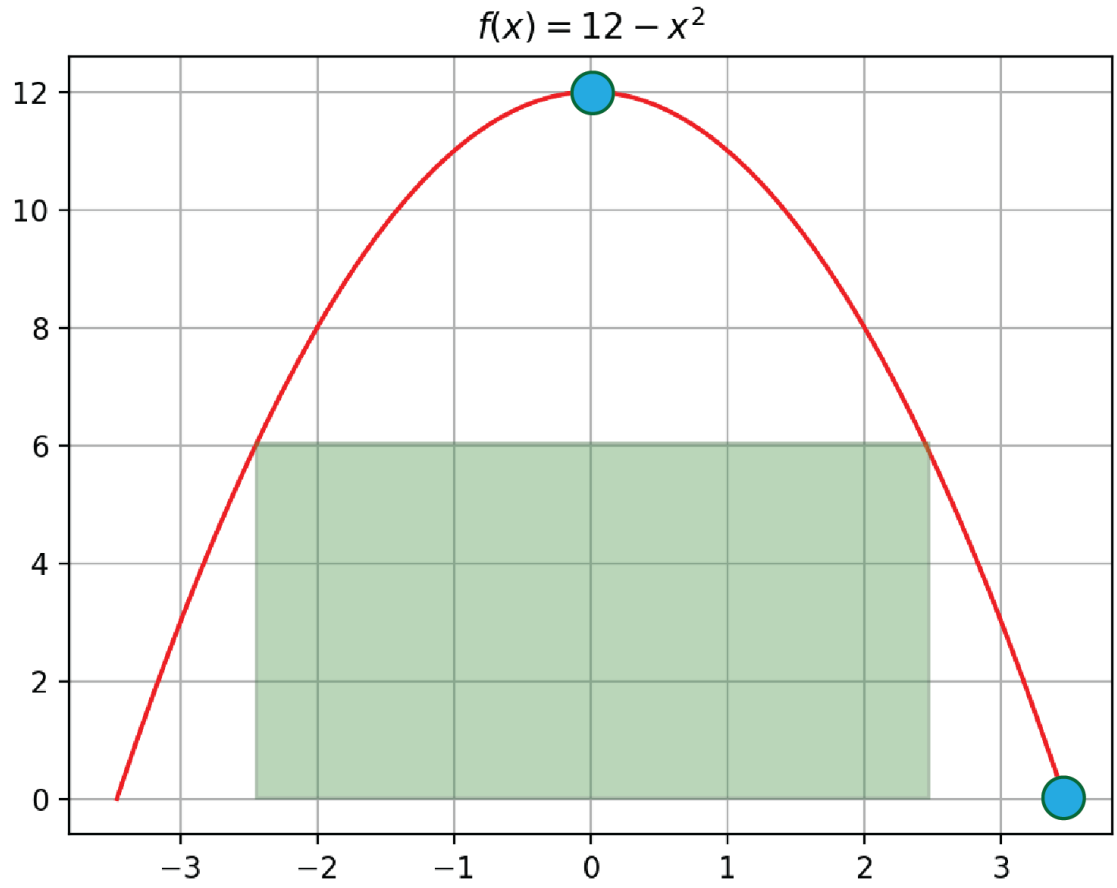

Figure 3: Largest area of a rectangle inscribed $f(x) \geq 0$ for $f(x) = 12 - x^2$.

11: Optimization

Observe Fig. 3. The function highlighted is $f(x) = 12 - x^2$ as the red line. We inscribe a rectangle for $f(x) \geq 0$. Find the dimensions of the largest rectangle possible. Observe that you only need to look at $x \in [0, 2\sqrt{3}]$, but make sure about the dimensions!

The following code:

```

a,b = 0,2 * math.sqrt(3)

def f(x):
    return 12 - x**2

print(op_rect(a,b,f))
```

produces

```
(4.0, 8.0)
```

Deliverables Problem 11

- Complete the functions.
- While we are using half of the interval $[0, 2\sqrt{3}]$ (please pay attention to that)
- *hint: don't use range if you want to save time*
- Round both dimensions to 2 decimal places.

12: Revenue Optimization

A company you've contracted for has the following models for price, revenue, cost, and profit, respectively:

$$p(x) = 5.00 - 0.002x \tag{41}$$
$$R(x) = xp(x) \tag{42}$$
$$C(x) = 3.00 + 1.10x \tag{43}$$
$$P(x) = R(x) - C(s) \tag{44}$$

To find the unit number to maximize P, we *could* take a derivative $\frac{dP}{dx}$ and solve the critical point. On the other hand we can look through the space of solutions (to find the peak or maximum profit). The following code:

```

p_,u_ = max_revenue(R,1,1000)
print(u_)

# to make the graph
#x = np.linspace(1,1299,100)
#y = P(x)
#plt.plot(x,y,'r')
#plt.plot(u_,p_,'go')
#plt.xlabel("Units")
#plt.ylabel("Revenue $MM")
#plt.title ("Maximizing Revenue")
#plt.show()
```

produces

```
975
```

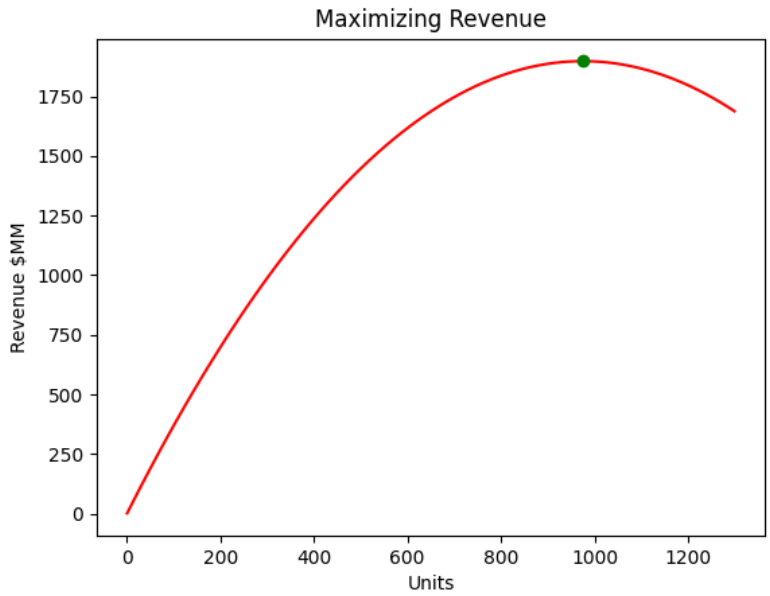

Figure 4: Plotting profit as function of units. The green dot, at 975, represents the maximal profit.

Deliverables Problem 12

- Complete the functions.
- Observe that the function returns a tuple (x, y) where x revenue and y are unit number.

Programming pairs

amgandhi@iu.edu, gravipat@iu.edu
craftdc@iu.edu, teswank@iu.edu
zaycarey@iu.edu, menonvi@iu.edu
ryandaun@iu.edu, bfmark@iu.edu
arkali@iu.edu, erugova@iu.edu
mbadwe@iu.edu, crschuit@iu.edu
rachguo@iu.edu, bsai@iu.edu
nkushnir@iu.edu, ptapiaro@iu.edu
shaagore@iu.edu, surehma@iu.edu
changali@iu.edu, khwillar@iu.edu
tkalale@iu.edu, lomwean@iu.edu
wchorich@iu.edu, amatariy@iu.edu
cdinman@iu.edu, benserna@iu.edu
jvgottum@iu.edu, mitcchar@iu.edu
jadebail@iu.edu, ashahi@iu.edu

tkonkim@iu.edu, rsreedh@iu.edu
eberens@iu.edu, wagnebr@iu.edu
jaxkatz@iu.edu, msantell@iu.edu
conkeen@iu.edu, jamread@iu.edu
sgudapur@iu.edu, mvsimi@iu.edu
balian@iu.edu, ctsvetan@iu.edu
jkchaura@iu.edu, sparab@iu.edu
adevill@iu.edu, stewti@iu.edu
nbkotur@iu.edu, zstopher@iu.edu
nabatja@iu.edu, pratmodi@iu.edu
josajack@iu.edu, tryerrag@iu.edu
chenkay@iu.edu, ggriorda@iu.edu
aadkhan@iu.edu, javitto@iu.edu
hedavid@iu.edu, reddysk@iu.edu
edjgarci@iu.edu, ceprenti@iu.edu
boding@iu.edu, auspiege@iu.edu
pafrost@iu.edu, mxrobins@iu.edu
cgreywal@iu.edu, tiffyou@iu.edu
cfahy@iu.edu, cmm23@iu.edu
reafrank@iu.edu, jianlou@iu.edu
reychaud@iu.edu, chsabir@iu.edu
jailjohn@iu.edu, ap59@iu.edu
cbremner@iu.edu, nrimawi@iu.edu
chodgman@iu.edu, smunikun@iu.edu
logellis@iu.edu, wmccarte@iu.edu
wbrogan@iu.edu, kammich@iu.edu
arkhal@iu.edu, archshar@iu.edu
agunde@iu.edu, lineri@iu.edu
emiellis@iu.edu, frvisco@iu.edu
belleli@iu.edu, arangi@iu.edu
andhodge@iu.edu, snlnu@iu.edu
spebrook@iu.edu, jlopezmo@iu.edu
kimyouh@iu.edu, arstover@iu.edu
jhbruce@iu.edu, saravula@iu.edu
jabbarke@iu.edu, camlueck@iu.edu
achitred@iu.edu, haoztian@iu.edu
sdharmmi@iu.edu, joebrahm@iu.edu
aydclink@iu.edu, iopfings@iu.edu
kylrguth@iu.edu, leoett@iu.edu
renahan@iu.edu, tmukund@iu.edu
lbadredd@iu.edu, njvinson@iu.edu

gmhowell@iu.edu, nvoneper@iu.edu
sjbarry@iu.edu, ishetty@iu.edu
kespejo@iu.edu, nmetlapa@iu.edu
adeglur@iu.edu, twirges@iu.edu
okorkmaz@iu.edu, animerch@iu.edu
cedonica@iu.edu, breramer@iu.edu
harrisdg@iu.edu, jmatczak@iu.edu
sambereb@iu.edu, adpulipa@iu.edu
marbelli@iu.edu, vupatel@iu.edu
benitezb@iu.edu, abbalokh@iu.edu
askaul@iu.edu, manushah@iu.edu
allennam@iu.edu, jusshah@iu.edu
auconner@iu.edu, colbstap@iu.edu
pkurzawa@iu.edu, peterabl@iu.edu
burkhel@iu.edu, jocpoiri@iu.edu
blevidut@iu.edu, gstogdil@iu.edu
kayfaris@iu.edu, mgstack@iu.edu
zcarmich@iu.edu, fshamrin@iu.edu
rcerauli@iu.edu, msolomie@iu.edu
akumares@iu.edu, forlin@iu.edu
cc96@iu.edu, olipfing@iu.edu
lasdodd@iu.edu, shmulla@iu.edu
sebcont@iu.edu, ntorpoco@iu.edu
ocrowder@iu.edu, bpovinel@iu.edu
ecanant@iu.edu, corrosar@iu.edu
blberger@iu.edu, brshelby@iu.edu
abelani@iu.edu, rtripat@iu.edu
daveanu@iu.edu, avaweish@iu.edu
beckab@iu.edu, etmillho@iu.edu
madimond@iu.edu, peterant@iu.edu
camehugh@iu.edu, rimaeda@iu.edu
jakedeck@iu.edu, nicmill@iu.edu
ejbusch@iu.edu, johnsanc@iu.edu
anchinna@iu.edu, traepena@iu.edu
nsfried@iu.edu, bwmoore@iu.edu
jamlian@iu.edu, wangjinz@iu.edu
ml82@iu.edu, cmoverfe@iu.edu
njustin@iu.edu, katepate@iu.edu
cjduffie@iu.edu, aquintas@iu.edu
careyjo@iu.edu, mackeyr@iu.edu
jdemian@iu.edu, rahkpate@iu.edu

zlangley@iu.edu, nsenthi@iu.edu
mgrass@iu.edu, ksreejit@iu.edu
allentp@iu.edu, syeshala@iu.edu
bcabral@iu.edu, ymarrakc@iu.edu
shbivens@iu.edu, wardjohn@iu.edu
harmonnc@iu.edu, amoza@iu.edu
presharr@iu.edu, shubtiwa@iu.edu
ykassahu@iu.edu, camyee@iu.edu
ankotel@iu.edu, andmuchi@iu.edu
eynkansa@iu.edu, amtsmith@iu.edu
anudivek@iu.edu, trimedio@iu.edu
langgrav@iu.edu, masoroki@iu.edu
nkhai@iu.edu, xsoto@iu.edu
ecabebe@iu.edu, weinstab@iu.edu
berrynm@iu.edu, lawlin@iu.edu
samjimen@iu.edu, alexpan@iu.edu
lrborows@iu.edu, joprayit@iu.edu
rkeel@iu.edu, jomalek@iu.edu
markdeng@iu.edu, smremmer@iu.edu
chamblia@iu.edu, nsmurthy@iu.edu